Checkpoint C — Talking points (MyCampus)

Opening line (if needed)

"Our user is an international student who has just arrived and asks *where is this room?* The guided search ends with building + floor + room. We checked with a test that the stored floor becomes the right label, and we showed that the test catches an off-by-one floor. What we do **not** know yet is whether a foreign student reads "Floor 1" the way we mean it, and whether the place data is real: it is still sample data, and our on-campus moderators have not checked it yet."

1. Which risk matters most, and why

The main risk: **the student ends up in the wrong place** (wrong building, floor or room). The app looks like it works, so nobody notices, and the student pays for it (late for a lecture or an exam).

Five-risk map (one control is not a test):

Failure	Probability	Impact	Detectability	Control	Residual risk
Wrong place data (floor, room, building)	High: `seed.sql` holds " (esempio)" sample data and coordinates are marked "indicative"	Medium-high: student lost or late	Low: no signal	**Non-test (detect):** our moderators, who are physically on campus, check every place on site after a developer enters it	The gap between entry and the moderator's check: the schema has no "verified" flag, so unchecked places are already shown to students; places that change later (a vending machine moved)
Floor label misread	Plausible: "ground floor = 0" vs "ground = 1st floor" (US, China…)	Medium: one floor off	Low	team decision on the label format	User interpretation not checked
Step 2 lists a building without the service, or hides one	Unknown	Medium: dead end "No places found"	Medium	Integration test against local Supabase (proposed, **not run yet**)	Query unverified today

Failure	Probability	Impact	Detectability	Control	Residual risk
Backend unreachable (free Supabase project paused after 1 week, no network on arrival)	Medium	High: app gives no answer	High: error screen	Contain: `AsyncView` with "Try again"; recover: migrations + seed rebuild the DB	No offline mode; technical error message
Unauthorized edits to place data: once moderators can write, a wrong access policy (RLS) lets anyone with the app's public key change or delete places	Low today: RLS is on and every table is read-only (select only); rises when moderator editing is added	High: every student can be sent to the wrong place	Low: a malicious edit looks like real data	Test (detect): with the public key, an insert or update on places must be refused; review checks every new policy	Moderator accounts themselves: a mistaken or stolen account can still edit

2. What we tested and what happened

- **Claim:** a student who follows the guided search to a place card is shown the right building, floor and room. This needs two kinds of evidence: the **data** must be true, and the **app** must show the stored data correctly.
- **Evidence 1 — the data (moderators, non-test):** after a developer enters a place, a moderator goes there in person and checks building, floor, room and description. Record: `[place, moderator, date, outcome — to fill]`.
- **Evidence 2 — the app (automated tests):**
  `bun run check` runs lint, `tsc --noEmit` and Jest.
  The test for this claim is `src/utils/format-floor.test.ts`: the stored floor becomes the label the student sees (0 → "Ground floor", 2 → "Floor 2", −1 → "Basement 1", unknown → nothing).
- **Observed result:** lint and types OK; 4 suites, 15 tests passed.
- **Does the test catch a defect?** We deliberately injected an off-by-one floor (`Floor ${floor + 1}`): the test fails with *Expected "Floor 2", Received "Floor 3"*. With the code restored, 3/3 pass.
- **Why we need both:** the tests cannot tell if a floor in the database is wrong; the moderator cannot tell if the code turns a correct floor into a wrong label. Each one covers the other's blind spot.

- **How the evidence could mislead:** the test's expected "Floor 2" comes from the same draft as the code, and the moderators are on our campus and read floors with the Italian convention too. Both confirm the data and the code (**verification**); neither shows that an international student reads "Floor 2" as we mean it (**validation**): for an American student, "2nd floor" is our first floor.
- **Evidence boundary:** not covered: the step 2 query, the real screen on a phone, how a real international student reads the label, and places that change after the moderator's check.

3. What review changed

- **Change reviewed:** a place entered by a developer. `[link to the PR or issue — to fill]`
- **Reviewer independence:** the moderator does not compare the entry with what the developer wrote, but with the real building on site. That basis can contradict the author; a second developer reading the same row could not.
- **Main question:** "Is this place really where the entry says: building, floor, room?"
- **Decision:** accept or revise `[outcome — to fill]`. On revise, the developer fixes the data and the moderator checks again.
- **Second review (on the floor-label test):** "Where does the expected 'Floor 2' come from?" Decision *seek context*: the team decides the label format after trying it with 1–2 international students, then the test is updated to the agreed rule.

4. AI use

- **Yes.** Automated checks with AI: it ran `bun run check`, did the injected-defect check and drafted the risk map.
- **Human-owned decision:** which risk comes first, the label format, and accept or revise on each place (the moderators).
- **Independent verification:** we rerun `bun run check` ourselves; the places are checked on site by the moderators, not by the AI; we reread the test against the team's decision, not against the code.
- **Corrected or rejected output:** the "Floor 2" oracle generated together with the code is not accepted as justification: it stays "to be validated". The first draft's data control ("a second person checks on site") was replaced with our real moderator process.

5. What remains uncertain and what comes next

- **Justified confidence:** the number → label conversion is correct for the three tested cases, and an off-by-one floor is detected. Nothing more.
- **Main unknown:** whether the data is real and whether foreign students read the floor as we intend.
- **Next control:** moderators check the real places on campus + trying the label with 1–2 international students; then the step 2 integration test.
- **Definition of Done change:** "A place is shown to students only after a moderator has checked it on campus; a change is merged only when `bun run check` passes." This turns the moderator check from detect into prevent. Cost: one moderator visit per place, a "verified" flag in the schema, and running the check before every merge.